VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



Deep Learning and Its Applications (CO3133)

ASSIGNMENT 2

Five-Stage Sleep Classification from EEG

Milestone 2: Draft Report

Instructor: Dr. Le Thanh Sach

Students: Nguyen Hoang Quoc – 2353027
Nguyen Vu Long – 2352696
Le Phuoc Vu – 2353341

Ho Chi Minh City, 2026

Table of Contents

List of Figures **2**

List of Tables **3**

I Introduction **4**
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Objectives and Milestone Scope 4
1.4 Proposed Approach and Current Status 4

II Dataset and Exploratory Analysis **5**
2.1 Dataset Description and Approval 5
2.2 Annotation Mapping and Eligible Epochs 5
2.3 Exploratory Analysis of the Full Cohort 6
2.4 Participant-Covering Subsampling 8
2.5 Data Splitting and Leakage Prevention 8
2.6 Waveform Preprocessing and Export 9

III Methodology **11**
3.1 Shared Data and Workflow 11
3.2 Custom CNN Baseline: StandardCNN2D 11
3.3 Custom Transformer 11
3.4 Pretrained EEG Foundation Model: CBraMod 12
3.5 Additional Residual-Attention Comparator 13
3.6 Learning Objective and Training Procedure 13
3.7 Evaluation Measures and Comparison Design 14
3.8 Implementation Attribution and Reproducibility 15

IV Preliminary Experimental Results **15**
4.1 Completed Runs and Overall Performance 15
4.2 Learning Behavior and Checkpoint Selection 16
4.3 Per-Stage Performance and Initial Error Analysis 17
4.4 Model Size and Computational Evidence 17

V Discussion, Limitations and Remaining Work **18**
5.1 What the Current Evidence Supports 18
5.2 Limitations 18
5.3 Work Planned for M3 18

VI Conclusion **19**

AI Usage Disclosure **19**

A Evidence and Report Completion Notes **19**

References **21**

List of Figures

1 Measured stage proportions before and after participant-level subsampling, and within the sampled development/test partitions. Subsampling preserves coverage but does not preserve exact original proportions. 7

2 Reproduced raw EEG and log-PSD preview for a labelled N2 epoch from development participant SC400. Only Fpz-Cz is plotted. This unfiltered EDA view is separate from the filtered waveform training input. 7

3 Actual preprocessed development waveforms: the first stored example of each stage, with recording and onset shown. Both EEG channels are plotted in the stored units. These are input examples, not model-prediction explanations. 10

4 Shared workflow. Softmax supplies metric probabilities; argmax supplies stage predictions. No temporal smoothing is applied. 11

5 Fold-1 loss and macro-F1 reconstructed from saved epoch logs. Training metrics aggregate predictions made during parameter updates; validation metrics use evaluation mode after the epoch. 16

6 Pooled selected-validation confusion matrices over 5,478 development epochs. Cell text gives counts; colour shows the fraction within each true-stage row. 17

List of Tables

1 Five-class mapping shared by EDA, preprocessing and training. 5
2 Counts from the exploration CSV and the current waveform export. The sampled total includes the reserved test set. 6
3 Reconstructed grouped splits for the current sampled dataset. All five classes occur in each training and validation partition. 8
4 Settings stored in the supplied result notebooks. The incomplete CNN variant also requests dropout 0.6, which the current local CNN constructor does not accept. 13
5 Two checkpoint-selection choices for the draft study. Both use development validation data, not the reserved test set. 14
6 Preliminary five-fold validation results at AUROC-selected checkpoints. No test results are included. Sources: the four completed notebooks listed in Appendix A. 15
7 Main CBraMod per-stage metrics recomputed from the pooled validation confusion matrix. These pooled metrics are not the five-fold averages in Table 6. 17
8 Current code parameter counts and historical notebook timing evidence. Epoch timers include training, validation and metric calculation; they exclude setup, model loading, plotting and preprocessing. 18
9 Local evidence sources. Notebook filenames are identifiers, not claims of final evaluation status. 20

I Introduction

1.1 Background and Motivation

Sleep-stage classification assigns a stage to a short interval of a physiological recording. This project studies whether representations learned by a pretrained EEG model can improve five-stage classification when the available training budget is small. A compact convolutional network provides the main reference; a custom Transformer provides an additional architecture comparison. The task is retrospective sleep staging, rather than sleep-disorder diagnosis or a validated clinical application.

1.2 Problem Statement

The prediction unit is one non-overlapping, labelled 30-second EEG epoch from two channels. After preprocessing, an example is

$$X_i \in \mathbb{R}^{2\times 30\times 200}, \qquad y_i \in \{0,1,2,3,4\} = \{\mathrm{W}, \mathrm{N1}, \mathrm{N2}, \mathrm{N3}, \mathrm{REM}\}. \tag{1}$$

A classifier produces five logits, with the predicted stage given by their argmax. The central question is: *Does fine-tuned CBraMod improve stage-balanced performance over a custom CNN when both use the same sampled waveform epochs and participant-separated development protocol?* Difficulties include unequal stage frequencies, variation between participants, and a substantial reduction in the number of epochs retained for training.

1.3 Objectives and Milestone Scope

The objectives are to document the data and leakage controls, construct a reproducible waveform pipeline, establish a CNN baseline, and evaluate pretrained CBraMod. The local `Transformer` is a further scratch-trained comparator whose performance remains to be measured.

This is an M2 progress report. Section 7.5 of the handbook asks for a simple baseline, a pretrained model in progress, EDA, and preliminary metrics; M2 contributes 25% of Assignment 2. The complete controlled study, expanded error analysis and final submission package belong to M3. Sections 20–22 also identify accuracy and macro-F1 as classification measures and require compute-cost evidence for the completed assignment [1]. This draft reports existing validation runs and explicitly identifies unfinished work.

1.4 Proposed Approach and Current Status

The original proposal explored EEG spectrograms. The present experiments instead give all models the same waveform tensor. CBraMod supplies pretrained EEG features;

`StandardCNN2D` and `ResCBAM_EEG_Net` are trained from scratch. Four notebook configurations have complete five-fold summaries, while one CNN variant is incomplete. The held-out test participants are excluded from the training notebooks and no test-performance result is claimed here.

II Dataset and Exploratory Analysis

2.1 Dataset Description and Approval

The source is Sleep-EDF Database Expanded, PhysioNet version 1.0.0, DOI `10.13026/C2X676`, distributed under the Open Data Commons Attribution License v1.0 [2, 3, 4]. This project uses the Sleep Cassette cohort: 78 participants and 153 recordings, excluding the separate Sleep Telemetry study. The source provides PSG waveforms in EDF and expert hypnograms in EDF+. The selected EEG derivations are Fpz-Cz and Pz-Oz, recorded at 100 Hz. They are the two EEG channels available in this cohort; other recorded signals are not additional EEG channels.

The dataset proposal records instructor approval without conditions. For M2, the accepted experimental protocol uses the 6,474-epoch waveform subset and participant-grouped cross-validation. This report documents that current protocol separately from the earlier full-cohort spectrogram proposal.

2.2 Annotation Mapping and Eligible Epochs

Original sleep stages 3 and 4 are merged into N3. This is a conversion of the legacy Rechtschaffen–Kales labels, not a claim that the recordings were rescored under a newer standard. Movement and unscored annotations are excluded. An annotation interval can supply several complete epochs, so annotation rows must not be counted as classification samples.

Index	Target	Source annotation
0	W	Sleep stage W
1	N1	Sleep stage 1
2	N2	Sleep stage 2
3	N3	Sleep stage 3 or 4
4	REM	Sleep stage R

Table 1: *Five-class mapping shared by EDA, preprocessing and training.*

The analysis retains complete, non-overlapping 30-second epochs between 30 minutes before the first scored sleep interval and 30 minutes after the last, subject to recording boundaries. This limits long wake periods. Because the rule uses expert labels, it defines a retrospective benchmark; it does not reproduce an untrimmed deployment setting.

2.3 Exploratory Analysis of the Full Cohort

2.3.1 Inventory, integrity and coverage

The annotation exploration in `sleep_exploration.ipynb` verified 153 hypnograms and obtained 195,469 eligible candidate epochs. At that notebook's saved execution, PSG downloads were not yet available and its preview cell failed. The later completed waveform export records archive-checksum checks for all 153 PSG/hypnogram pairs, timing and boundary checks, and finite-signal checks. These are different stages of evidence, not a claim that the initial notebook completed every signal check.

The current export contains all 78 participants and all 153 recordings. Integrity checks detect incomplete or inconsistent data, but the pipeline does not perform comprehensive physiological artefact rejection. No claim is made that every finite EEG epoch is artefact-free.

2.3.2 Stage distribution and imbalance

Table 2 distinguishes the full eligible population from the sampled data actually used in the experiments. In the eligible population, N2 is 35.37% and N3 is 6.67%, a count ratio of about 5.30:1. In sampled development data, N2 is 34.88% and N3 is 7.90%, a ratio of about 4.41:1. These measured values motivate macro-averaged and per-stage metrics. They do not support assuming that N2 occupies 45–55% of this dataset.

Stage	Full eligible	Full (%)	Sampled total	Development	Test
W	65,941	33.73	1,900	1,674	226
N1	21,522	11.01	770	677	93
N2	69,132	35.37	2,310	1,911	399
N3	13,039	6.67	545	433	112
REM	25,835	13.22	949	783	166
Total	195,469	100.00	6,474	5,478	996

Table 2: *Counts from the exploration CSV and the current waveform export. The sampled total includes the reserved test set.*

2.3.3 Signal length and exploratory visualization

Each original EEG epoch has shape $(2, 3000)$, representing two channels over 30 seconds at 100 Hz. The original EDA used a two-second Hann window with a 0.5-second hop, retained 0.5–30 Hz, and took log power spectral density, yielding $(2, 60, 57)$. Figure 2 reproduces that exploratory view from a checksum-verified raw recording. It explains the earlier representation choice; it is not the tensor supplied to the current models.

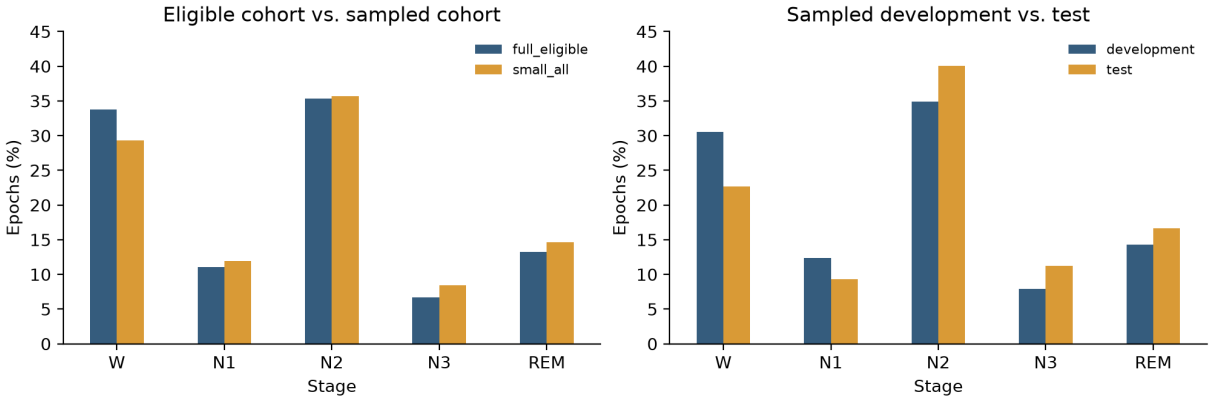


Figure 1: *Measured stage proportions before and after participant-level subsampling, and within the sampled development/test partitions. Subsampling preserves coverage but does not preserve exact original proportions.*

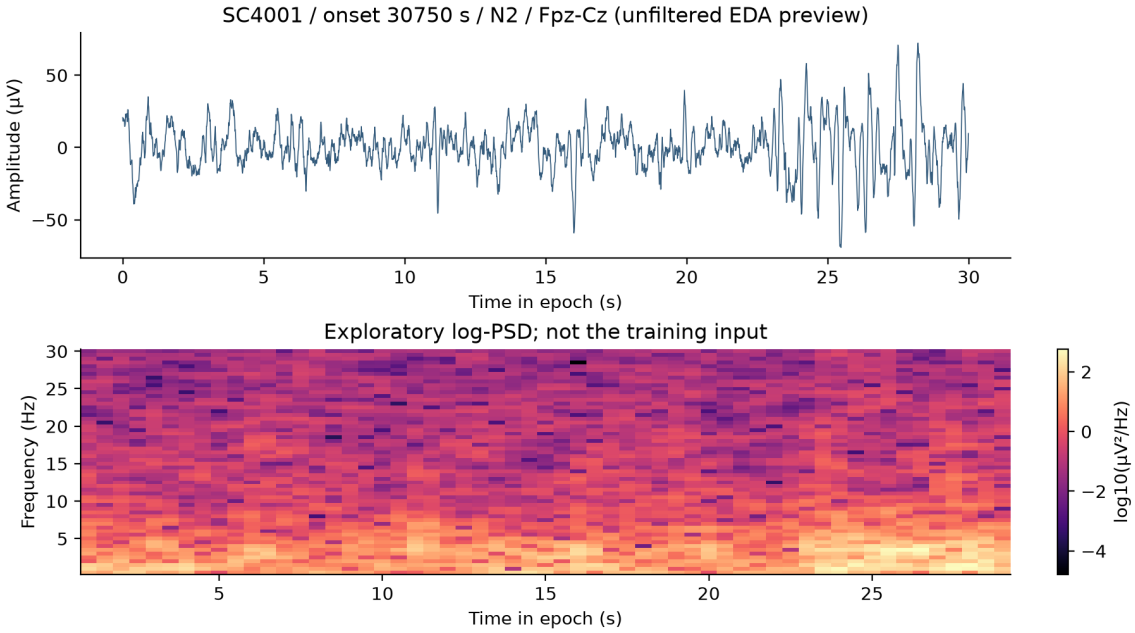


Figure 2: *Reproduced raw EEG and log-PSD preview for a labelled N2 epoch from development participant SC400. Only Fpz-Cz is plotted. This unfiltered EDA view is separate from the filtered waveform training input.*

2.4 Participant-Covering Subsampling

Training on all 170,537 eligible development epochs exceeded the available compute budget. The current `preprocessing.py` therefore selects up to 83 epochs per participant directly from EDF annotations, before waveform patch extraction. Every available night–stage combination receives at least one epoch; the remaining quota is allocated proportionally to its available count, using largest fractional remainders. Sampling uses seed 42 without replacement. In this cohort, every participant supplies 83 epochs, giving $78 \times 83 = 6,474$.

This strategy retains participant and night coverage while reducing storage and training work. It is neither class balancing nor preservation of the full night's temporal sequence. Its label-stratified selection also applies to the reserved test subset, so eventual results on those 996 epochs must be described as performance on a sampled benchmark, not on all held-out-night epochs.

2.5 Data Splitting and Leakage Prevention

Twelve participants are reserved for testing; the remaining 66 form the development set. All epochs and nights from one participant stay in the same partition. The original test membership was obtained by shuffling sorted participant IDs with Python seed 42 and reserving the last 12. The exporter retains these assignments and the original fold metadata.

The supplied training notebooks, however, *regenerate* five development folds with `StratifiedGroupKFold`, shuffle enabled and random state 42, using the sampled labels. Consequently, their folds must not be described as the original full-data folds in `cv_folds.csv`. The reconstructed sampled-fold class counts agree with every completed run's confusion-matrix supports. Exact runtime participant manifests were not saved by those notebooks and should be exported before the final comparison.

Fold(s)	Train people	Val. people	Train epochs	Val. epochs
1–4, each	53	13	4,399	1,079
5	52	14	4,316	1,162
Reserved test	12 participants		—	996

Table 3: *Reconstructed grouped splits for the current sampled dataset. All five classes occur in each training and validation partition.*

Accepted M2 data protocol. Preprocessing produces the mapped source tensor $(6474, 2, 30, 200)$. Of these 6,474 epochs, 5,478 belong to development and 996 are reserved for testing. StratifiedGroupKFold partitions the development epochs into the training and validation folds in Table 3; it does not create additional samples. The subset size and grouped-fold protocol have been accepted for this draft. The per-fold counts are reported to describe the actual experiments.

2.6 Waveform Preprocessing and Export

The implemented pathway in `preprocessing.py` is:

1. Pair each PSG with its hypnogram, check participant assignments, channel identity, sample rate, annotation timing, and complete epoch boundaries. Archive checksums and comparison with existing spectrogram epoch metadata run when those references are available; both were enabled in the current export.

2. Apply the stage mapping, wake margins and participant sampling rule. Preserve recording IDs and onsets for traceability.

3. Filter each continuous two-channel recording with a zero-phase 0.3–35 Hz FIR bandpass, then resample from 100 to 200 Hz with polyphase resampling. Filtering precedes extraction of the selected patches. Resampling matches the backbone interface and does not create new measured high-frequency information.

4. Convert MNE's volts to units of $100\,\mu$V by multiplying by 10^4. Extract 6,000 samples per channel and reshape in temporal order to $(2, 30, 200)$: 30 one-second patches per channel.

5. Save float32 waveforms, int64 labels, participant/recording IDs, onsets, split/fold metadata, configuration and provenance. Temporary recording shards are merged into `dataset/sleep_waveform_small.pt`, accompanied by its selection CSV.

The final file occupies approximately 311 MB, compared with about 9.39 GB for the earlier full waveform tensor; local raw PSGs occupy about 7.60 GB. CPU preprocessing works recording by recording. The current pipeline applies no additional population z-score, augmentation, notch filter, or amplitude-threshold artefact rejection. `get_stats` is an optional per-channel diagnostic over both time axes, not an active normalization step.

An ordinary full batch contains float32 inputs $(32, 2, 30, 200)$ and int64 targets $(32,)$. The network returns logits $(32, 5)$. The last batch may contain fewer than 32 epochs. The two axes of the waveform grid represent seconds and within-second samples; neither axis represents frequency.

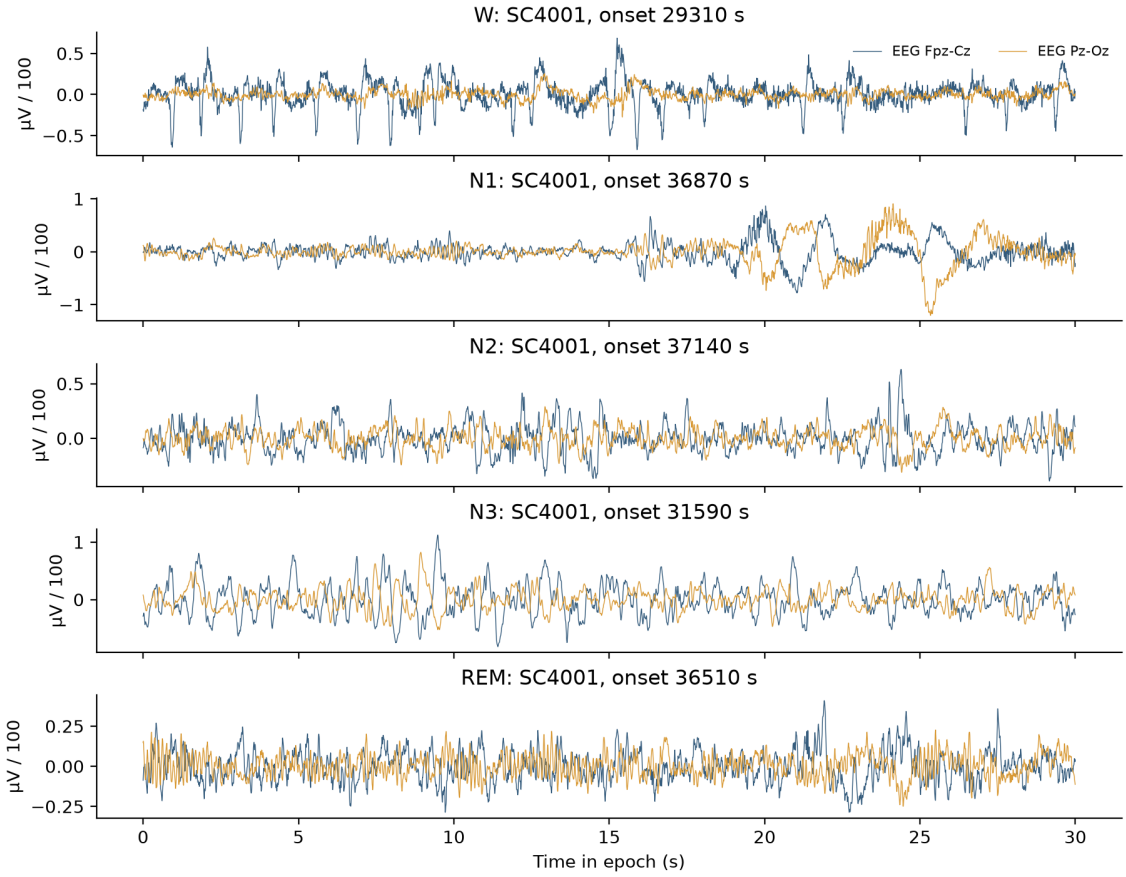


Figure 3: *Actual preprocessed development waveforms: the first stored example of each stage, with recording and onset shown. Both EEG channels are plotted in the stored units. These are input examples, not model-prediction explanations.*

III Methodology

3.1 Shared Data and Workflow

`EEGDataset` memory-maps the saved tensor on CPU and exposes only development indices, retaining the metadata needed for participant grouping. DataLoaders transfer batches to the selected device. `ApplyTransform` currently acts as an identity transformation, preserving the export's amplitude scale.

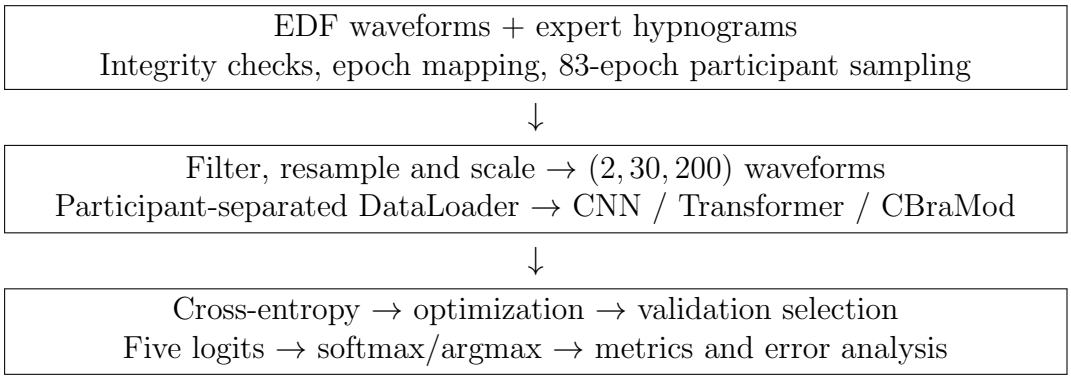


Figure 4: *Shared workflow. Softmax supplies metric probabilities; argmax supplies stage predictions. No temporal smoothing is applied.*

3.2 Custom CNN Baseline: StandardCNN2D

The main scratch baseline contains four 3×3 convolution blocks, each followed by batch normalization, ReLU and 2×2 max pooling. Channel widths are 32, 64, 128 and 256. For a $(2, 30, 200)$ input, the successive feature shapes are $(32, 15, 100)$, $(64, 7, 50)$, $(128, 3, 25)$ and $(256, 1, 12)$. Global average pooling is followed by a $256 \to 512 \to 5$ classifier with ReLU and dropout 0.5.

The current implementation has 523,237 trainable parameters. It learns local patterns on the waveform grid without pretrained weights. Its imposed two-dimensional neighbourhood mixes positions within and across seconds; it should not be described as convolution on a time–frequency spectrogram. Pooling also limits detailed temporal localization.

3.3 Custom Transformer

The local `Transformer` uses a shared three-block CNN to encode each EEG channel's entire waveform grid. Adaptive pooling and a projection produce one 256-dimensional token per channel. A learned classification token and positional embeddings produce *three tokens* for this two-channel task. Four encoder layers use eight-head self-attention and feed-forward width 768, followed by a normalized classification token and a five-logit head.

There are 2,640,149 trainable parameters in the current default model. This is a

scratch-trained channel-token Transformer, not a pretrained vision Transformer and not CBraMod's 60-patch architecture. It can model relationships between channel summaries, while temporal detail has already been compressed by its CNN encoder. Shape compatibility has been checked; no trained Transformer result is present in the supplied experiment notebooks.

3.4 Pretrained EEG Foundation Model: CBraMod

3.4.1 Pretraining and input adaptation

CBraMod was introduced by Wang et al. as an EEG foundation model pretrained on the Temple University Hospital EEG corpus using masked patch reconstruction [5]. This project reuses the released checkpoint rather than conducting foundation-model pretraining. The current input has two channels, 30 one-second segments per channel, and 200 values per segment, giving 60 EEG patches per epoch. The scaling follows the upstream convention of expressing EEG in units of $100\,\mu$V [6]. Our 0.3–35 Hz Sleep-EDF filtering remains a project-specific preprocessing choice.

3.4.2 Patch embedding and criss-cross Transformer

In the local `cbramod.py`, each patch is represented by a temporal convolution branch and a frequency branch. The latter computes the magnitude of a 200-point real FFT, obtains 101 coefficients, and projects them to 200 dimensions. Their embeddings are added, followed by a depthwise convolutional positional encoding with kernel $(19, 7)$. Thus the external input is waveform data even though the network internally derives spectral features.

The local `criss_cross_transformer.py` contains 12 encoder layers with hidden width 200 and feed-forward width 800. Each layer divides the feature dimension into two halves. Four attention heads model relationships *across channels at a fixed second*; another four model relationships *across the 30 seconds within a channel*. Their outputs are concatenated and passed through residual and feed-forward operations. This preserves the channel–time structure and differs from flattening all patches into a single attention sequence.

3.4.3 Classification head and fine-tuning

`CBraModClassifier` strictly loads the pretrained backbone, replaces the reconstruction projection with an identity, and uses a new $12,000 \to 800 \to 200 \to 5$ head with ELU and dropout 0.5. The inspected runs use `freeze_encoder=False`, so the head and backbone are jointly updated. Patch embedding, including the FFT, remains float32 when the rest of the CUDA training path uses automatic mixed precision.

The current task-specific model has 14,645,605 trainable parameters, plus 200 fixed mask parameters. This count includes the large flattening classifier and should not be substi-

tuted with an upstream backbone-only parameter count. Two-channel bipolar recordings also differ from the pretraining montage; successful checkpoint loading alone does not prove useful transfer.

3.5 Additional Residual-Attention Comparator

`ResCBAM_EEG_Net` uses four residual blocks of widths 32/64/128/256 with channel and spatial attention, global pooling and a $256 \to 64 \to 5$ head. Its current configuration has 471,565 trainable parameters. It provides an additional scratch-model result, while `StandardCNN2D` remains the simple baseline. Its different optimizer and learning rate mean that its present comparison is not a controlled test of attention alone.

3.6 Learning Objective and Training Procedure

For mutually exclusive stages, the models minimize unweighted multiclass cross-entropy:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(z_{i,y_i})}{\sum_{c=0}^{4} \exp(z_{i,c})}. \tag{2}$$

Targets are integer class indices and the models return raw logits. The inspected runs use batch size 32, at most 80 epochs, cosine scheduling with $T_{\max} = 100$ and minimum learning rate 10^{-6}, and seeds 42 for NumPy, PyTorch and grouped splitting. Each fold constructs a new model and optimizer. CUDA AMP is enabled when CUDA is selected.

Configuration	Optimizer	Initial LR	Weight decay
StandardCNN2D	Adam	3×10^{-5}	10^{-4}
CBraMod, main	Adam	3×10^{-5}	10^{-4}
CBraMod, LR variant	Adam	10^{-5}	10^{-4}
ResCBAM	AdamW	10^{-3}	5×10^{-4}
CNN variant, incomplete	Adam	10^{-5}	10^{-4}

Table 4: *Settings stored in the supplied result notebooks. The incomplete CNN variant also requests dropout 0.6, which the current local CNN constructor does not accept.*

Selection rule for the results below: the five supplied result notebooks save the in-memory state with highest validation macro one-vs-rest AUROC, and early-stop on validation loss with patience 10 and minimum improvement 0.001. All metrics in a fold's summary come from that AUROC-selected epoch. At this draft stage, AUROC and Macro-F1 remain two checkpoint-selection choices under investigation; the final choice is not yet fixed. The existing notebooks restore weights in memory but do not persist a trained checkpoint per fold.

3.7 Evaluation Measures and Comparison Design

Accuracy and macro-F1 satisfy the classification metric pair specified by the handbook. Macro-F1 is the main stage-balanced reporting measure:

$$\text{Macro-F1} = \frac{1}{5}\sum_{c=0}^{4}\frac{2P_cR_c}{P_c + R_c}. \tag{3}$$

The implementation also reports macro precision, macro recall, macro one-vs-rest AUROC from softmax probabilities, unweighted Cohen's kappa, and confusion matrices. Macro recall equals balanced accuracy for this five-class single-label task. Macro-F1 is the mean of class-wise F1 scores, not the harmonic mean of the two macro averages.

3.7.1 Checkpoint-Selection Choices Under Investigation

The draft considers the two validation-based choices in Table 5. AUROC evaluates probability ranking across classes; Macro-F1 directly evaluates stage-balanced argmax predictions. A checkpoint with higher AUROC need not give higher Macro-F1, so the choice is an experimental question rather than a settled conclusion.

Selection criterion	Motivation	M2 status
Validation macro AUROC	Select strong one-vs-rest probability ranking.	Used by the recorded runs; preliminary results available.
Validation Macro-F1	Select balanced class-wise precision and recall for stage predictions.	Alternative to evaluate; no completed comparison reported here.

Table 5: *Two checkpoint-selection choices for the draft study. Both use development validation data, not the reserved test set.*

To isolate checkpoint selection, compare both criteria on the same training trajectory within each fold, keeping the model, initialization, data order, optimizer, epoch budget and loss-based stopping rule fixed. Retain both selected states and compare their validation accuracy, Macro-F1, AUROC, kappa and per-stage errors. Whether stopping should also follow Macro-F1 is a separate training-policy choice. The final protocol will be chosen using development evidence before evaluating the reserved test participants.

3.7.2 Model and Learning-Rate Comparisons

The principal hypothesis is that fine-tuned CBraMod gives higher validation macro-F1 than the scratch CNN. Inputs, grouping settings, labels and evaluation definitions are shared; model capacity, architecture and pretraining differ. Therefore this comparison evaluates complete models and does not isolate the effect of pretraining.

A second, preliminary factor comparison changes CBraMod's initial learning rate from 10^{-5} to 3×10^{-5}. The two saved notebook sources differ only in that value. The decision measure is mean validation macro-F1, with accuracy and variability also reported.

However, seeds are set once before the CV loop: different earlier stopping lengths can change later folds' random initialization and data order. A stronger paired study should reset the same seed for each corresponding fold and retain runtime code/configuration snapshots. The planned final ablation is frozen-backbone versus full fine-tuning with the same checkpoint, head, epochs, folds and preprocessing; no frozen-model result is claimed yet.

3.8 Implementation Attribution and Reproducibility

The project-specific components are EEG preprocessing/export, sampling, dataset loading, task heads, scratch architectures and experiment adaptation in `model_lib.py`. CBraMod's backbone and criss-cross attention are reused/adapted upstream implementations [6], not original algorithms proposed by this group. PyTorch supplies neural-network and optimization operations; MNE and SciPy supply signal processing; scikit-learn supplies splitting and metrics.

The saved outputs record CUDA use, but do not identify GPU model or peak memory. Their version display predates a scikit-learn reinstall, so a post-install runtime version snapshot is still needed. Current local architecture counts are documented separately from historical run provenance: the notebooks do not embed the exact `model_lib.py` used remotely. These limits should be resolved before a strict final reproducibility or efficiency claim.

IV Preliminary Experimental Results

4.1 Completed Runs and Overall Performance

Table 6 transcribes complete notebook summaries. Values are unweighted means across five development folds, with the population standard deviation used by the notebooks (`np.std`, ddof = 0). They describe fold variability, not confidence intervals or independent repeated-seed trials. Accuracy, precision, recall, F1 and kappa were independently recomputed from all 24 available completed-fold confusion matrices and agreed within displayed rounding. AUROC cannot be reconstructed from hard-label confusion matrices and is taken from the recorded probability-based logs.

Configuration	Accuracy	Macro-F1	AUROC	Kappa
StandardCNN2D	$.6965 \pm .0226$	$.5927 \pm .0204$	$.8972 \pm .0103$	$.5781 \pm .0292$
CBraMod, 3×10^{-5}	$.7550 \pm .0295$	$.6747 \pm .0230$	$.9226 \pm .0144$	$.6648 \pm .0377$
CBraMod, 10^{-5}	$.7247 \pm .0496$	$.6282 \pm .0719$	$.9158 \pm .0220$	$.6220 \pm .0676$
ResCBAM	$.6907 \pm .0328$	$.5602 \pm .0371$	$.8944 \pm .0125$	$.5664 \pm .0469$

Table 6: *Preliminary five-fold validation results at AUROC-selected checkpoints. No test results are included. Sources: the four completed notebooks listed in Appendix A.*

The main CBraMod run exceeds the CNN by 0.0585 accuracy and 0.0820 macro-F1, or

5.85 and 8.20 percentage points. This supports the model-comparison hypothesis for these recorded configurations, without establishing statistical significance or a benefit caused solely by pretraining. The higher CBraMod learning rate also has a 0.0465 higher mean macro-F1 than the lower-rate run; the pairing limitations in Section III prevent a clean causal conclusion.

`training_CNN2D_0-6.ipynb` contains four completed folds and a fifth fold logged through epoch 54, with no completed fifth-fold summary. It is excluded from the five-fold table. Its requested `dropout=0.6` argument also requires recovery of the model version used for that run. No Transformer score is available, so it is not assigned a placeholder numerical result.

4.2 Learning Behavior and Checkpoint Selection

For the main CBraMod run, selected epochs were 15, 14, 25, 15 and 15; training ended at epochs 25, 24, 28, 25 and 25. The CNN selected epochs 20, 27, 22, 12 and 20. Figure 5 shows fold 1 from each run. Continued improvement in training loss together with deteriorating validation loss is consistent with overfitting. The dashed line denotes selection by AUROC, which need not coincide with peak macro-F1 or minimum validation loss.

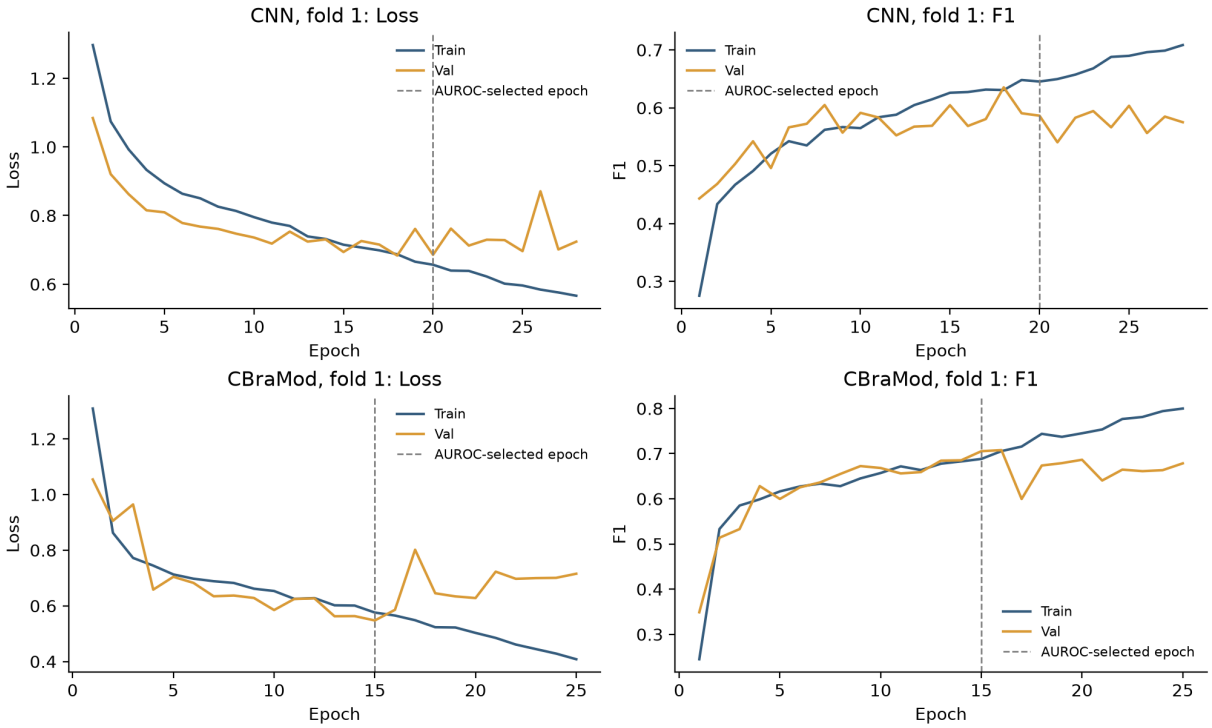


Figure 5: *Fold-1 loss and macro-F1 reconstructed from saved epoch logs. Training metrics aggregate predictions made during parameter updates; validation metrics use evaluation mode after the epoch.*

4.3 Per-Stage Performance and Initial Error Analysis

Figure 6 sums the five selected-validation confusion matrices for each model. These pooled counts are distinct from averaging fold-level metrics. Validation labels were used for checkpoint selection, so the figure is development error analysis rather than an independent test estimate.

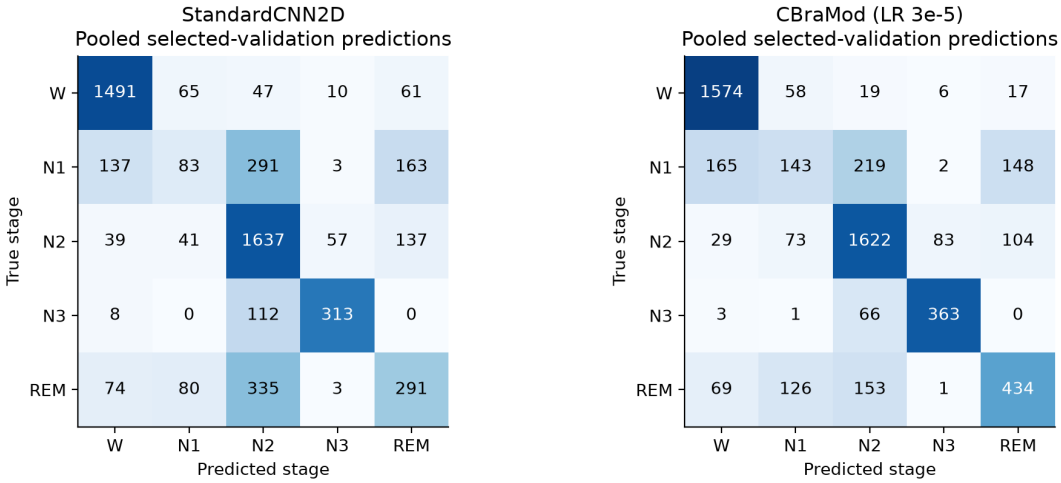


Figure 6: *Pooled selected-validation confusion matrices over 5,478 development epochs. Cell text gives counts; colour shows the fraction within each true-stage row.*

Stage	Support	Precision	Recall	F1
W	1,674	.8554	.9403	.8958
N1	677	.3566	.2112	.2653
N2	1,911	.7802	.8488	.8130
N3	433	.7978	.8383	.8176
REM	783	.6174	.5543	.5841

Table 7: *Main CBraMod per-stage metrics recomputed from the pooled validation confusion matrix. These pooled metrics are not the five-fold averages in Table 6.*

N1 and REM remain the main weaknesses. Of 677 true N1 epochs, 219 are predicted as N2 (32.35%); of 783 REM epochs, 153 are predicted as N2 (19.54%). N1 and REM together account for 883 of 1,342 errors. N3 is less frequent than N1 but is classified substantially better, so class frequency alone does not explain the observed failures. Stage transitions, ambiguous patterns and the absence of neighbouring-epoch context are hypotheses for further investigation, not causes established by these counts.

Traceable correctly classified, difficult-correct and misclassified waveform examples remain to be exported with recording/onset IDs. The EDA examples in Figure 3 do not substitute for that prediction-level qualitative analysis.

4.4 Model Size and Computational Evidence

The timer sums are provisional observations, not matched-hardware speed benchmarks. GPU identity, inference latency and peak memory were not saved. The final report should

Model	Current trainable parameters	Logged epochs	Sum of epoch timers (s)
StandardCNN2D	523,237	132	79.44
CBraMod, main	14,645,605	127	713.42
ResCBAM	471,565	187	305.79
Transformer	2,640,149	—	—

Table 8: *Current code parameter counts and historical notebook timing evidence. Epoch timers include training, validation and metric calculation; they exclude setup, model loading, plotting and preprocessing.*

measure these on the same hardware with explicit batch size, precision, warm-up and timing boundaries. Current parameter counts likewise require matching to archived runtime model definitions before being treated as exact counts for the historical runs.

V Discussion, Limitations and Remaining Work

5.1 What the Current Evidence Supports

The implementation can turn raw EEG into a small participant-covering waveform dataset, train scratch and pretrained classifiers, and produce stage-aware validation metrics. The main CBraMod configuration has stronger recorded development performance than the simple CNN, but N1 recall remains low. Neither pretrained status nor added attention guarantees improvement: the additional ResCBAM configuration does not outperform the simpler CNN here.

5.2 Limitations

The 83-epoch quota sacrifices within-participant coverage and temporal continuity. The sampled benchmark, two EEG derivations, legacy labels and label-dependent wake trimming limit generalization. Zero-phase filtering uses surrounding samples within the recording and therefore does not demonstrate causal online inference. Participant separation addresses identity leakage, but does not remove selection optimism from reusing validation for checkpoint and configuration choices. One seed stream, incomplete checkpoint provenance, no held-out evaluation and unmatched cost evidence further limit conclusions.

5.3 Work Planned for M3

1. Retain the accepted subset and participant separation, and archive exact fold membership and epoch identities for every model.

2. Complete the custom Transformer comparison if budget allows. Prioritize a controlled frozen-backbone versus full-fine-tuning study over adding more architectures. In the frozen condition, keep the frozen encoder in evaluation mode as well as disabling its gradients.

3. Compare AUROC-based and Macro-F1-based checkpoint choices on development data and document the chosen final rule. Keep checkpoint selection distinct from the early-stopping policy, reset seeds for corresponding folds, and save weights, configurations, histories and validation probabilities.

4. Export prediction-level examples and investigate the N1/REM errors; record hardware, runtime environment, inference latency and peak memory.

5. Select the final configuration using development results only, refit on development with a CV-derived duration, and evaluate the reserved test participants after decisions are fixed. Complete the final report, slides, video, assignment page and reproducibility links.

VI Conclusion

This M2 study establishes a waveform preprocessing pipeline and preliminary grouped validation evidence for five-stage sleep classification on the accepted 6,474-epoch subset. Fine-tuned CBraMod currently improves recorded accuracy and macro-F1 over Standard-CNN2D, while stage-specific errors and protocol limitations remain. The next stage will compare checkpoint-selection choices, strengthen the controlled comparison and reproducibility, and provide held-out evaluation.

AI Usage Disclosure

OpenAI Codex assisted with code adaptation/debugging in the project and with this draft's structure, source inspection, figure/table extraction and wording. Numerical summaries were traced to saved notebook outputs and dataset metadata; the report preparation did not conduct new training. The authors remain responsible for reviewing every claim, confirming runtime provenance, and completing the outstanding experiments and contribution statement. Task details are recorded in the repository's `AI_USAGE.md`.

A Evidence and Report Completion Notes

The accompanying `results_by_fold.csv`, `results_summary.csv` and `evidence.json` retain extracted results and source hashes. `build_figures.py` reproduces the report figures and checks the logged hard-label metrics without training. The report's README maps its sections to the handbook and lists remaining publication details. Group identification, repository/page links, checkpoint access and individual contributions must be completed by the authors before publication; this draft does not invent those records.

Source	Use in this draft
`sleep_exploration.ipynb`	Initial annotation EDA; its saved PSG-preview error is distinguished from later verification.
`preprocessing.py`	Current direct EDF-to-waveform sampling and export.
`training_final.ipynb`	CBraMod, LR 3×10^{-5}; five completed folds.
`training_CNN2D.ipynb`	StandardCNN2D baseline; five completed folds.
`training_cbra-Copy1.ipynb`	CBraMod, LR 10^{-5}; five completed folds.
`training_ResCBAM.ipynb`	Additional scratch comparator; five completed folds.
`training_CNN2D_0-6.ipynb`	Incomplete alternative; excluded from the five-fold summary.
`model_lib.py`	Current architectures and locally counted parameters.

Table 9: *Local evidence sources. Notebook filenames are identifiers, not claims of final evaluation status.*

References

[1] "CO3133: Deep Learning and Its Applications – Course Project Handbook," 2026, supplied handbook-ene-v2.pdf. Sections 3, 7.5, 7.7, and 19–23. M2 schedule: printed p. 16; Assignment 2 rubric: printed p. 27.

[2] B. Kemp, "Sleep-EDF Database Expanded," 2013, version 1.0.0; expanded archive described in 2018. Accessed 28 October 2026. [Online]. Available: https://physionet.org/content/sleep-edfx/1.0.0/

[3] B. Kemp, A. H. Zwinderman, B. Tuk, H. A. C. Kamphuisen, and J. J. L. Oberye, "Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG," *IEEE Transactions on Biomedical Engineering*, vol. 47, no. 9, pp. 1185–1194, 2000. [Online]. Available: https://ieeexplore.ieee.org/document/867928/

[4] T. Pollard, B. E. Moody, L.-w. H. Lehman, B. Gow, C. Fernandes, C. Xie, A. Johnson, R. G. Mark, and T. Heldt, "PhysioNet as a global platform for biomedical research," *Nature Health*, vol. 1, no. 8, pp. 792–795, 2026. [Online]. Available: https://doi.org/10.1038/s44360-026-00096-z

[5] J. Wang, S. Zhao, Z. Luo, Y. Zhou, H. Jiang, S. Li, T. Li, and G. Pan, "CBraMod: A criss-cross brain foundation model for EEG decoding," in *The Thirteenth International Conference on Learning Representations*, 2025, author manuscript: https://arxiv.org/abs/2412.07236. [Online]. Available: https://openreview.net/forum?id=NPNUHgHF2w

[6] J. Wang *et al.*, "CBraMod: Official implementation and pretrained model," 2025, mIT-licensed source; downstream ISRUC loader uses division by 100. Local adaptations: cbramod.py, criss_cross_transformer.py and model_lib.py. Accessed 28 October 2026. [Online]. Available: https://github.com/wjq-learning/CBraMod